

Hướng Dẫn Từng Bước: lsb-secret-header

HUONG DAN TUNG BUOC: LAB GIAU TIN BANG LSB + HEADER BI MAT

Tên bài: lsb-secret-header

QUAN TRỌNG: Khi mới mở lab, sẽ CHƯA CÓ encoded.png và recovered.txt.

- encoded.png chỉ được tạo sau khi bạn sửa xong encode.py và chạy lệnh encode.
- recovered.txt chỉ được tạo sau khi bạn sửa xong decode.py và chạy lệnh decode.
- check_work.sh cũng sẽ xóa file kết quả cũ rồi tạo lại để kiểm tra.

RAW URL cài lab:

<https://github.com/vuxvinh/KiThuatGiauTin/raw/main/lsb-secret-header.tar>

Chạy:

```
imodule https://github.com/vuxvinh/KiThuatGiauTin/raw/main/lsb-secret-header.tar
labtainer lsb-secret-header
```

Nếu muốn làm lại từ đầu:

```
labtainer -r lsb-secret-header
```

Kiểm tra file ban đầu:

```
ls
```

Bạn sẽ thấy các file:

```
README.md
cover.png
no_secret.png
secret.txt
encode.py
decode.py
check_work.sh
make_assets.py
```

Nếu không thấy encoded.png hoặc recovered.txt lúc này thì DUNG. Hai file đó chưa được tạo.

Chạy:

```
bash check_work.sh
```

Lúc chưa sửa code, kết quả sẽ FAIL. Đây là bình thường.

Lý do:

- encode.py còn TODO nên chưa tạo encoded.png.
- decode.py còn TODO nên chưa tạo recovered.txt.

- analysis.txt chưa được tạo.

Mở file:

```
nano encode.py
```

Trong encode.py, tìm hàm build_packet(payload) và thay toàn bộ phần thân hàm bằng:

```
def build_packet(payload: bytes) -> bytes:
    length_bytes = struct.pack(">I", len(payload))
    digest = hashlib.sha256(payload).digest()
    return MAGIC + length_bytes + digest + payload
```

Tìm hàm capacity_bits(img) và thay bằng:

```
def capacity_bits(img: Image.Image) -> int:
    return img.width * img.height * 3
```

Tìm hàm embed_packet(img, packet) và thay bằng:

```
def embed_packet(img: Image.Image, packet: bytes) -> Image.Image:
    img = img.convert("RGB")
    pixels = list(img.getdata())
```

```
channels = []
```

```
for r, g, b in pixels:
```

```
    channels.extend([r, g, b])
```

```
bits = list(bytes_to_bits(packet))
```

```
if len(bits) > len(channels):
```

```
    raise ValueError("Packet is too large for this image")
```

```
for i, bit in enumerate(bits):
```

```
    channels[i] = (channels[i] & 0xFE) | bit
```

```
new_pixels = []
```

```
for i in range(0, len(channels), 3):
```

```
    new_pixels.append((channels[i], channels[i + 1], channels[i + 2]))
```

```
out = Image.new("RGB", img.size)
```

```
out.putdata(new_pixels)
```

```
return out
```

Lưu file trong nano:

- Nhấn Ctrl+O
- Nhấn Enter

- Nhấn Ctrl+X

Kiểm tra encode.py:

```
python3 encode.py cover.png secret.txt encoded.png
```

Nếu đúng, sẽ thấy ENCODE_OK và lúc này mới có file encoded.png.

Kiểm tra:

```
ls -l encoded.png
```

Mở file:

```
nano decode.py
```

Tìm hàm image_lsb_bits(img) và thay bằng:

```
def image_lsb_bits(img: Image.Image):  
    img = img.convert("RGB")  
    for r, g, b in img.getdata():  
        yield r & 1  
        yield g & 1  
        yield b & 1
```

Tìm hàm bits_to_bytes(bits) và thay bằng:

```
def bits_to_bytes(bits):  
    data = bytearray()  
  
    for i in range(0, len(bits), 8):  
        chunk = bits[i:i + 8]  
  
        if len(chunk) < 8:  
            break  
  
        value = 0  
        for bit in chunk:  
            value = (value << 1) | bit  
  
        data.append(value)  
  
    return bytes(data)
```

Tìm hàm read_bytes_from_lsb(img, nbytes) và thay bằng:

```
def read_bytes_from_lsb(img: Image.Image, nbytes: int) -> bytes:  
    need_bits = nbytes * 8  
    bits = []  
  
    for bit in image_lsb_bits(img):
```

```
bits.append(bit)

if len(bits) == need_bits:
    break

if len(bits) < need_bits:
    raise ValueError("Image does not contain enough LSB bits")

return bits_to_bytes(bits)
```

Luu file trong nano:

- Nhan Ctrl+O
- Nhan Enter
- Nhan Ctrl+X

Kiem tra decode.py:

```
python3 decode.py encoded.png recovered.txt
```

Neu dung, se thay:

```
HEADER_OK
SHA256_OK
DECODE_OK
```

Luc nay moi co file recovered.txt.

Kiem tra:

```
ls -l recovered.txt
cat recovered.txt
```

So sanh voi secret.txt:

```
cmp secret.txt recovered.txt
echo $?
```

Neu echo \$? in ra 0 thi hai file giong nhau.

Chay:

```
python3 decode.py no_secret.png recovered_empty.txt
```

Ket qua dung:

```
NO_SECRET_HEADER
```

Ghi chu:

- Lenh nay co the thoat voi ma loi 3.

- Điều quan trọng là output có NO_SECRET_HEADER.
- Không cần tạo recovered_empty.txt.

Tạo file:

```
nano analysis.txt
```

Nhập dung 4 dòng sau:

```
capacity_bits=196608
capacity_bytes=24576
header_bytes=44
max_payload_bytes=24532
```

Lưu file:

- Nhấn Ctrl+O
- Nhấn Enter
- Nhấn Ctrl+X

Kiểm tra:

```
cat analysis.txt
```

Giai thích:

- Ảnh cover.png có kích thước 256x256.
- Ảnh RGB có 3 kênh màu R, G, B.
- Mỗi kênh gộp được 1 bit.
- $\text{capacity_bits} = 256 * 256 * 3 = 196608$.
- $\text{capacity_bytes} = 196608 / 8 = 24576$.
- $\text{header_bytes} = \text{MAGIC } 8 + \text{LENGTH } 4 + \text{SHA256 } 32 = 44$.
- $\text{max_payload_bytes} = 24576 - 44 = 24532$.

Chạy:

```
bash check_work.sh
```

Kết quả dung:

```
TASK1_ENCODE_IMAGE_PASS
TASK2_HEADER_DETECTED_PASS
TASK3_HASH_VALID_PASS
TASK4_RECOVERED_MATCH_PASS
TASK5_NEGATIVE_TEST_PASS
TASK6_CAPACITY_ANALYSIS_PASS
ALL_TASKS_PASS
```

Không có fail:

- Doc task nao FAIL.
- Xem file log tuong ung:

```
cat encode.log
cat decode.log
cat no_header.log
```

Chay:

```
checkwork lsb-secret-header
```

Ket qua dung:

```
Y - Task_1_Encode_image
Y - Task_2_Detect_secret_header
Y - Task_3_Verify_SHA256
Y - Task_4_Recover_original_message
Y - Task_5_Detect_image_without_secret
Y - Task_6_Calculate_capacity
Y - All_tasks_completed
```

Sau khi checkwork da pass, chay:

```
stoplab lsb-secret-header
```

File nop bai nam trong:

```
~/labtainer_xfer/lsb-secret-header
```

File can nop co dang:

```
<student_id>.lsb-secret-header.lab
```

1. Khong thay encoded.png

Binh thuong neu ban chua chay:

```
python3 encode.py cover.png secret.txt encoded.png
```

2. Khong thay recovered.txt

Binh thuong neu ban chua chay:

```
python3 decode.py encoded.png recovered.txt
```

3. encode.py bao NotImplementedError

Ban chua thay het cac ham TODO trong encode.py.

4. decode.py bao NotImplementedError

Ban chua thay het cac ham TODO trong decode.py.

5. HEADER fail

Co the do encode sai thu tu bit. Khi tach byte thanh bit phai di tu bit 7 ve bit 0.

6. SHA256 fail

Có thể do đọc sai LENGTH, sai endian, hoặc ghép byte sai thứ tự bit.

7. Negative test fail

Khi magic != MAGIC, phải print NO_SECRET_HEADER và thoát ngay.

8. Task 6 fail

Kiểm tra analysis.txt có đúng tên file, đúng 4 dòng, đúng giá trị và không viết sai key.

1. Kỹ thuật LSB là gì?

2. Vì sao ảnh sau khi giấu tin nhìn gần giống ảnh gốc?

3. Header PTITLSB1 dùng để làm gì?

4. Vì sao cần trường LENGTH?

5. SHA-256 trong bài lab có tác dụng gì?

6. Công thức tính dung lượng giấu tin tối đa của ảnh RGB là gì?

7. Ảnh 256x256 RGB giấu được tối đa bao nhiêu byte payload?

8. Kỹ thuật LSB có an toàn tuyệt đối không? Vì sao?